LiDAR Coordinate Transformation System

TECHNICAL ARCHITECTURE & IMPLEMENTATION SPECIFICATION

Executive Summary: This application performs high-precision batch processing and coordinate conversion of large LiDAR (.las) point cloud files. It transforms global UTM (Zones 34N/35N) or ETRS89 coordinates into Romania's national spatial reference system (**Stereo70** and **Marea Neagră 1975** normal heights).

1. Main Objective & Problem Statement

Standard LiDAR acquisition sensors and GIS software export spatial data using global geographic coordinate systems or UTM projections (e.g., WGS84 / ETRS89). However, infrastructure, surveying, and cadastre projects in Romania legally require spatial data to be delivered in the local **Stereo70** projection with **Marea Neagră 1975** orthometric heights.

The Core Challenge

- **Massive Data Volumes:** Point cloud files routinely contain 10M to 100M+ points, leading to severe RAM bottlenecks in standard script executions.
- **Complex Geodetic Math:** Precise conversions require 2D/1D B-spline grid shift interpolations over official national grid distortion matrices (.GRT files).

The Technical Solution

- **Hybrid Execution Engine:** Combines Python for GUI, multi-threading, and file parsing with an embedded Java runtime for high-speed mathematical operations.
- **Automatic Data Chunking:** Dynamically splits large point clouds into batches of 10 million points to optimize memory usage and performance.

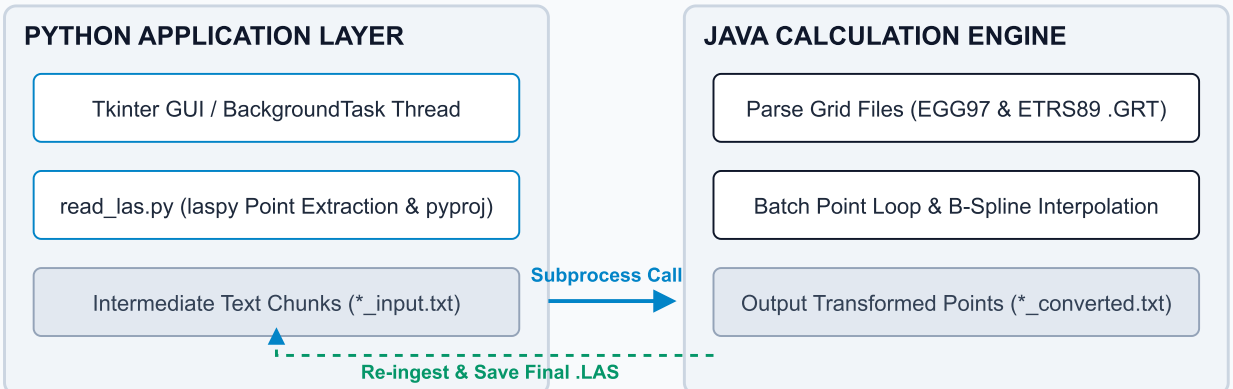
2. High-Level Project Structure

The system is organized into modular Python files, an embedded Java processing engine, and official national geodetic grid shift tables:

Component	Type	Role & Functionality
`main.py`	Python (Tkinter)	Desktop GUI, thread management, batch chunking, and Java subprocess coordination.
`read_las.py`	Python (laspy/numpy)	LAS file IO, header/VLR projection modification, raw point extraction, and file re-assembly.
`giveNewValues.java`	Java Source	Geodetic transformation engine. Originally built for single points, adapted to process multi-point files.
`jdk-14.0.2/`	JRE Directory	Bundled local Java runtime environment ensuring zero-dependency cross-machine execution.
`*.GRT Tables`	Binary Grid Files	Quasigeoid height matrix (`EGG97_QGR.GRT`) & 2D shift grid (`ETRS89_KRASOVSCHI42_2D.GRT`).

3. Architecture & Data Flow Diagram

The processing workflow spans Python UI operations, intermediate buffer file creation, external Java process execution, and final point cloud reconstruction:



4. Core Mathematical Implementation

The transformation pipeline implemented across Python and Java consists of three distinct mathematical stages:

Stage A: UTM/Geographic to ETRS89 Ellipsoidal Coordinates (Python)

Raw binary coordinates from the LAS file are scaled using header vectors ($X_{scaled} = X \cdot scale + offset$).

Python's `pyproj.Transformer` converts UTM Zone 34N/35N coordinates into ETRS89 geographic coordinates (Degrees, Minutes, Seconds):

```
      latD = int(latlong[0]) latM = int((latlong[0] - latD) * 60) latS = (latlong[0] -
latD - latM / 60) * 3600
```

Stage B: 2D Helmert & B-Spline Grid Interpolation (Java)

The Java engine projects ETRS89 coordinates onto an Oblique Stereographic projection and applies a 2D Helmert conformal transformation:

$$E_t = m \cdot (E_s \cdot cos(R_z) - N_s \cdot sin(R_z)) + t_E \quad | \quad N_t = m \cdot (N_s \cdot cos(R_z) + E_s \cdot sin(R_z)) + t_N$$

Exact local adjustments are computed using 16-node bicubic B-spline surface interpolation over local grid shifts extracted from `ETRS89_KRASOVSCHI42_2D.GRT`.

Stage C: Vertical Datum Quasigeoid Correction (Java)

Vertical conversion from ETRS89 ellipsoidal height (h) to Marea Neagră 1975 normal height (H_{MN}) utilizes the 1D interpolation module over the national quasigeoid model (`EGG97_QGR.GRT`):

$$H_{MN} = h - N_{quasigeoid}$$

5. Python-to-Java Integration & Code Adaptation

The Java transformation engine (`giveNewValues.java`) was originally an existing standalone routine designed for **single-point coordinate transformations**. To support high-volume point cloud processing, it required only slight modifications:

- **File Streaming Loop:** The execution flow was updated from reading isolated command-line parameters to iterating line-by-line over text input buffers containing hundreds of thousands of coordinate points.
- **Batch File Output:** Instead of printing a single transformed result to standard output, the code writes batch results sequentially into corresponding `*_converted.txt` files.

```
      # Python invocation passing input files to the adapted Java batch engine
subprocess.call( r'"{}\java" "{}\giveNewValues.java" 1 "{}" "{}"'.format( local_java_path,
working_dir, python_temp_input, working_dir ), shell=True )
```

6. Point Cloud Reconstruction & Verification

Once the Java subprocess terminates, Python validates the existence of the generated `*_converted.txt` file and executes the reconstruction pipeline inside `read_las.py`:

- **Header Bounding Calculation:** Reads converted Easting, Northing, and Height arrays to recalculate floor/ceiling bounds (X_{min}, Y_{min}, Z_{min} / X_{max}, Y_{max}, Z_{max}).
- **VLR GeoKey Updating:** Modifies standard GeoTIFF GeoKey directory tags inside the LAS file VLRs (setting `ProjectedCSTypeGeoKey = 3844` for Stereo70).
- **File Writing & Cleanup:** Writes out the formatted `.las` file, updates file timestamp metrics, and removes temporary working files.

System Resilience Note: The multi-threaded Tkinter UI remains fully responsive during computation. If a user cancels operation or an invalid LAS file is encountered, the worker thread safely halts the active background thread and preserves file integrity.